

Spring Boot 异常处理机制

1. 目标与原则

1.1. 面向相关方友好

1.2. 开发规范

2. 模块与数据结构设计

2.1. 异常类型枚举 ErrorCode

2.2. 统一响应体 Result<T>

2.3. 自定义异常 ServerException

3. 全局异常处理器

3.1. 关键思想

3.2. 实现

3.3. 业务层

3.4. 控制器

4. Spring Validation 与自定义注解

4.1. 校验和自定义异常结合

4.1.1. 添加依赖

4.1.2. 编写实体类

4.1.3. 控制器

4.1.4. 测试结果

4.2. 自定义注解

4.2.1. 编写自定义注解

4.2.2. 对应的校验器

4.2.3. 使用

1. 目标与原则

1.1. 面向相关方友好

- 后端：只做一件事——把自定义异常抛出，不关心页面跳转与响应结构。

- **前端**：后端**统一返回结构**与规范，接口风格一致。由**全局异常处理器**完成“异常 → 统一响应”的转换。
- **用户**：得到清晰可读的错误提示，接口请求返回统一的 JSON。

为什么要将运行时异常（`RuntimeException`）转为自定义异常？

`ConnectTimeoutException` 这类技术异常用户读不懂。转为自定义异常并在 `message` 中用**面向用户的语言**描述“发生了什么”，既减少沟通成本又避免泄露细节。

1.2. 开发规范

1. Controller/Service/DAO (Mapper) **不得截留异常**。只允许**手动抛出自定义异常**（`throw new ServerException(...)`），统一由全局处理器收口。
2. **统一使用 HTTP 状态码**：在团队约定集合内使用：`200`（成功）、`400`（用户输入错误）、`401`、`403`、`404`、`409`（数据冲突）、`500`（系统异常）、`999`（未知/保留）。
3. 自定义异常必须有 `message`：用**用户友好的语言**描述异常原因（不要技术术语）。
4. 禁止 `catch (Exception)` 这种**大一统捕获**；在业务代码中只在**必要边界**分小类捕获并转为自定义异常；统一兜底交给全局异常处理器。

2. 模块与数据结构设计

新建 `spring-boot-exception` 模块，新建 `enums`、`exception`、`result`、`handler`、`service`、`controller`、`annotation` 等子包。

2.1. 异常类型枚举 `ErrorCode`

坚持“枚举不超过 5~7 类”的记忆友好原则，使用 HTTP 码作为唯一编码，文案做**用户友好描述**。

```
1 @Getter
2 @AllArgsConstructor
3 public enum ErrorCode {
4     UNAUTHORIZED(401, "登录失效, 请重新登录"),
5     FORBIDDEN(403, "无权限访问"),
6     NOT_FOUND(404, "资源不存在"),
7     BAD_REQUEST(400, "请求参数不合法"),
8     CONFLICT(409, "数据冲突"),
9     INTERNAL_SERVER_ERROR(500, "服务器异常, 请稍后再试"),
10    UNKNOWN(999, "系统繁忙, 请稍后再试");
11
12    private final int code;
13    private final String msg;
14 }
```

若将来需要更细，不要扩枚举数量，而是在 `msg` 上做细化或在日志中补充“技术原因”，对外仍旧简洁。

2.2. 统一响应体 `Result<T>`

- `code`：对齐 HTTP 状态（200/400/500/...），易记；
- `msg`：面向用户的友好提示；
- `data`：业务数据；

```
1  @Data
2  public class Result<T> {
3      private int code = 0;
4
5      private String msg = "success";
6
7      private T data;
8
9      public static <T> Result<T> ok() {
10         return ok(null);
11     }
12
13     public static <T> Result<T> ok(T data) {
14         Result<T> result = new Result<>();
15         result.setData(data);
16         return result;
17     }
18
19     public static <T> Result<T> error() {
20         return error(ErrorCode.INTERNAL_SERVER_ERROR);
21     }
22
23     public static <T> Result<T> error(String msg) {
24         return error(ErrorCode.INTERNAL_SERVER_ERROR.getCode(), msg);
25     }
26
27     public static <T> Result<T> error(ErrorCode errorCode) {
28         return error(errorCode.getCode(), errorCode.getMsg());
29     }
30
31     public static <T> Result<T> error(int code, String msg) {
32         Result<T> result = new Result<>();
33         result.setCode(code);
34         result.setMsg(msg);
35         return result;
36     }
37 }
```

2.3. 自定义异常 `ServerException`

两个核心：范围受控的 code（源自 `ErrorCode`）、用户可读的 msg。

```
1  @Data
2  @EqualsAndHashCode(callSuper = true)
3  public class ServerException extends RuntimeException {
4      @Serial private static final long serialVersionUID = 1L;
5
6      private int code;
7      private String msg;
8
9      public ServerException(String msg) {
10         super(msg);
11         this.code = ErrorCode.INTERNAL_SERVER_ERROR.getCode();
12         this.msg = msg;
13     }
14     public ServerException(ErrorCode errorCode) {
15         super(errorCode.getMsg());
16         this.code = errorCode.getCode();
17         this.msg = errorCode.getMsg();
18     }
19     public ServerException(String msg, Throwable e) {
20         super(msg, e);
21         this.code = ErrorCode.INTERNAL_SERVER_ERROR.getCode();
22         this.msg = msg;
23     }
24 }
```

3. 全局异常处理器

3.1. 关键思想

API 请求 (Accept: application/json 或 X-Requested-With: XMLHttpRequest) → 统一返回 Result JSON;

3.2. 实现

```
1  @Slf4j
2  @RestControllerAdvice
3  public class GlobalExceptionHandler {
4
5      // ===== 自定义异常处理 =====
6      @ExceptionHandler(ServerException.class)
7      public Result<String> handleServerException(ServerException e) {
8          // 这里可以按需记录日志
9          log.warn("自定义异常: code={}, msg={}", e.getCode(), e.getMsg());
10         return Result.error(e.getCode(), e.getMsg());
11     }
12
13     // ===== 参数校验: @Valid @RequestBody 触发 (JSON 请求体) =====
14     @ExceptionHandler(MethodArgumentNotValidException.class)
15     public Result<String> handleMethodArgumentNotValid(MethodArgumentNotValidException ex) {
16         String msg = ex.getBindingResult().getFieldErrors().stream().findFirst().map(DefaultMessageSourceResolvable::getDefaultMessage).orElse("请求参数不合法");
17         return Result.error(400, msg);
18     }
19
20     // ===== 参数校验: 表单/路径绑定失败 (非 @RequestBody 场景) =====
21     @ExceptionHandler(BindException.class)
22     public Result<String> handleBindException(BindException ex) {
23         FieldError fe = ex.getFieldError();
24         String msg = (fe != null) ? (fe.getField() + " " + fe.getDefaultMessage()) : "请求参数不合法";
25         return Result.error(400, msg);
26     }
27
28     // ===== 单个参数校验: @RequestParam/@PathVariable 上的约束 (如 @Min) =====
29     @ExceptionHandler(ConstraintViolationException.class)
30     public Result<String> handleConstraintViolation(ConstraintViolationException ex) {
31         String msg = ex.getConstraintViolations().stream().findFirst().map(v -> v.getPropertyPath() + " " + v.getMessage()).orElse("请求参数不合法");
32         return Result.error(400, msg);
33     }
34
35     // ===== 请求体反序列化/类型不匹配等常见 400 =====
36     @ExceptionHandler({HttpMessageNotReadableException.class, MethodArgumentTypeMismatchException.class, MissingServletRequestParameterException.class})
```

```

37     public Result<String> handleBadRequest(Exception ex) {
38         return Result.error(400, "请求格式错误");
39     }
40
41     // ===== 兜底：未知异常 =====
42     @ExceptionHandler(Exception.class)
43     public Result<String> handleUnknown(Exception ex) {
44         log.error("未知异常: ", ex);
45         return Result.error(500, "服务器异常, 请稍后再试");
46     }
47 }

```

3.3. 业务层

```

▼ service 子包
Java |
1  @Service
2  public class ExceptionService {
3      public void unauthorizedError() {
4          // 未登录
5          throw new ServerException(ErrorCode.UNAUTHORIZED);
6      }
7
8      public void systemError() {
9          // 系统异常
10         throw new ServerException("系统异常");
11     }
12 }

```

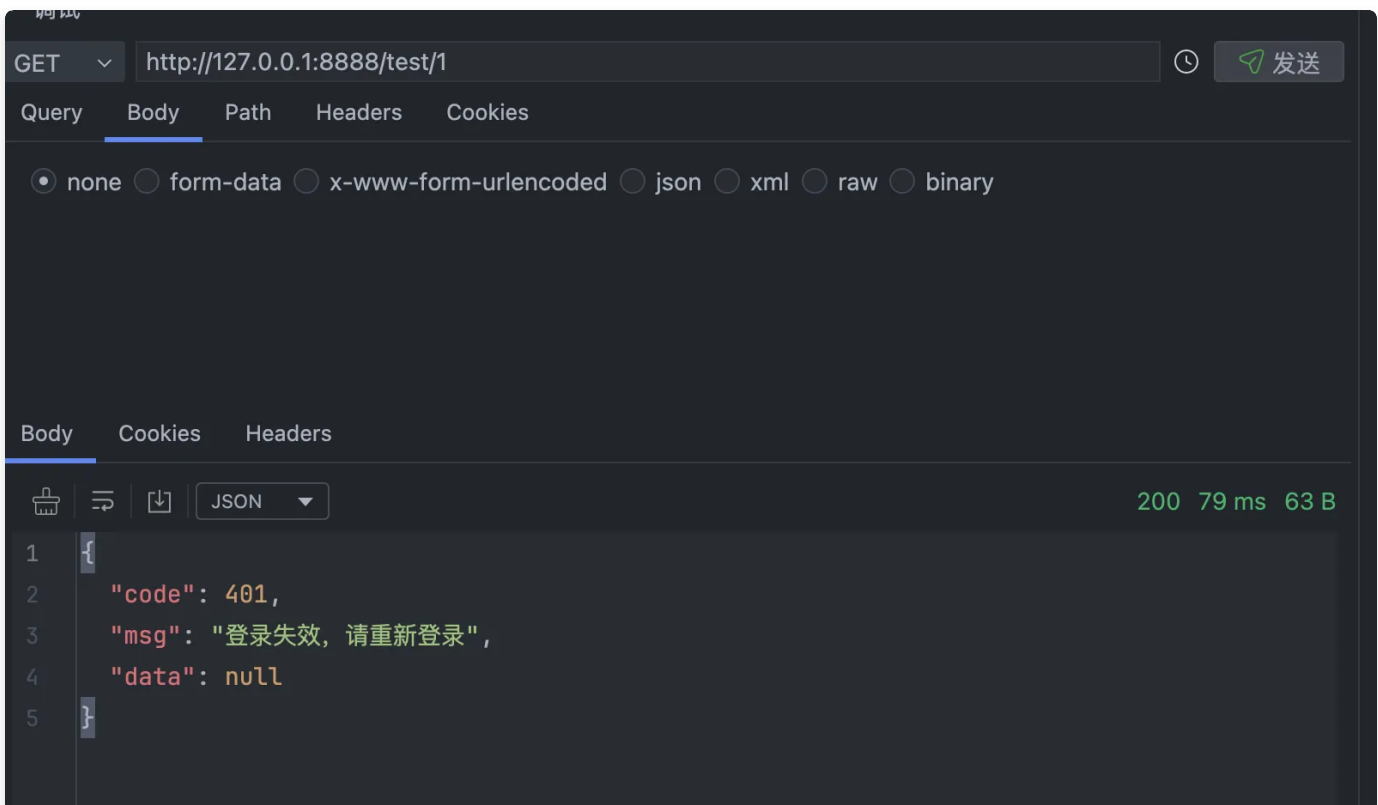
实战建议：

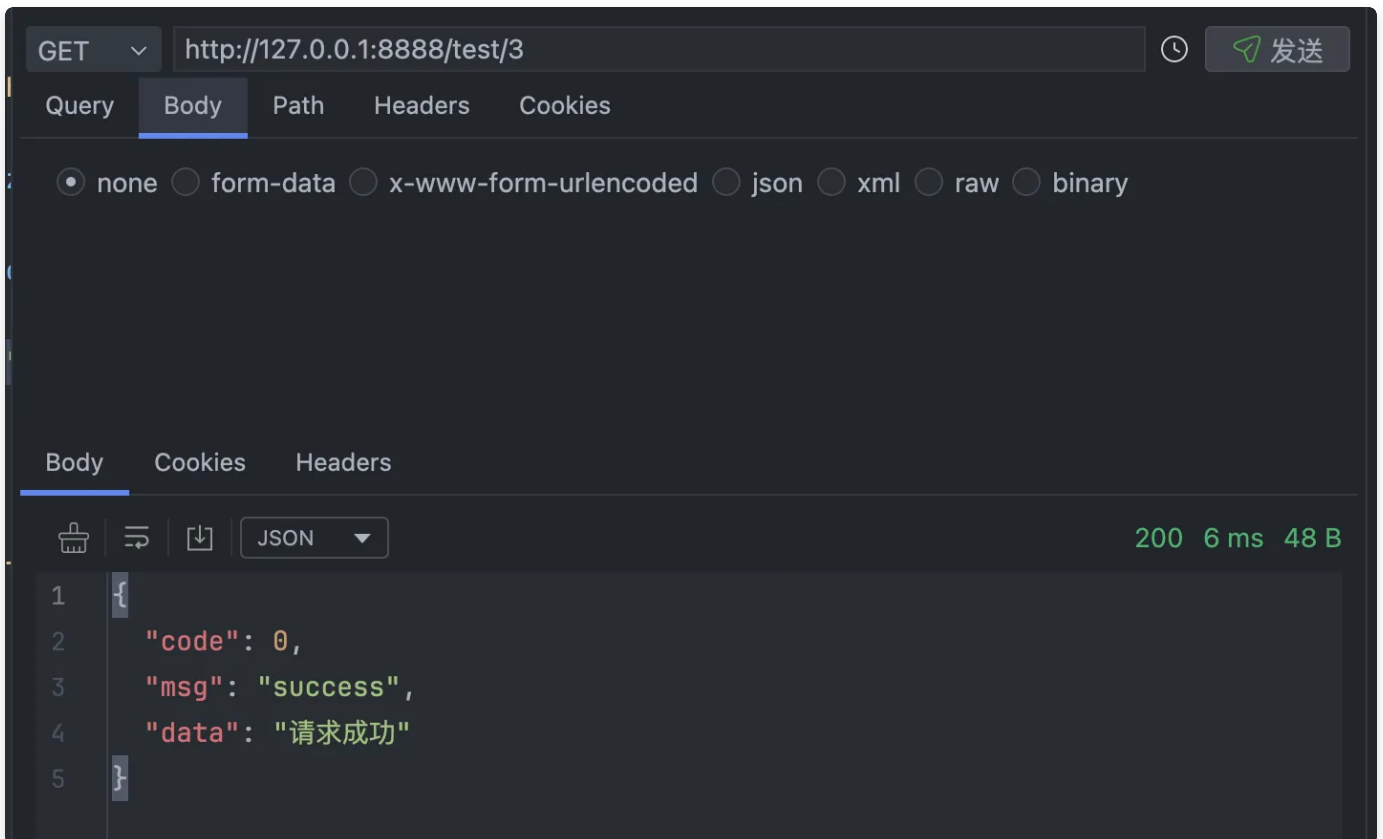
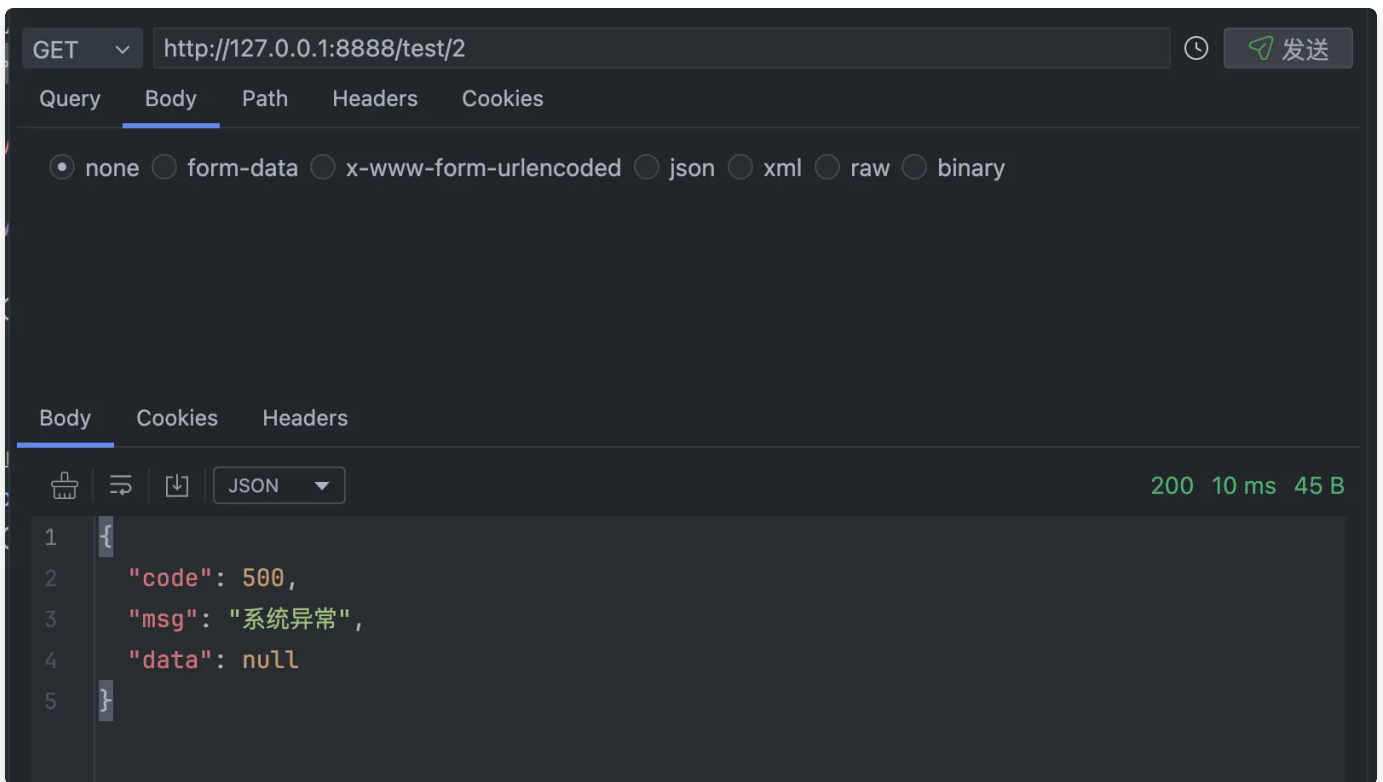
- 直接 `throw new ServerException(ErrorCode.UNAUTHORIZED);` 用具体错误枚举作为自定义异常参数；
- 或者 `throw new ServerException("订单不存在")` 这类的业务友好语。

3.4. 控制器

```
Java |
1  @RestController
2  public class ExceptionController {
3      @Resource private ExceptionService exceptionService;
4
5      @GetMapping("/test/{id}")
6      public Result<String> getInfo(@PathVariable int id) {
7          if (id == 1) {
8              exceptionService.unAuthorizedError();
9          } else if (id == 2) {
10              exceptionService.systemError();
11          }
12          return Result.ok("请求成功");
13      }
14  }
```

测试结果：





4. Spring Validation 与自定义注解

4.1. 校验和自定义异常结合

4.1.1. 添加依赖

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-validation</artifactId>
4 </dependency>
```

4.1.2. 编写实体类

```
1 @Data
2 public class User {
3     @NotBlank(message = "用户名不能为空")
4     private String username;
5
6     @NotBlank(message = "密码不能为空")
7     private String password;
8
9     @Max(value = 100, message = "年龄不能超过100岁")
10    @Min(value = 1, message = "年龄不能小于1岁")
11    private int age;
12
13    @NotBlank(message = "手机号不能为空")
14    private String phone;
15 }
```

4.1.3. 控制器

让校验异常自动抛出，由全局器统一处理，不再手写 `BindingResult`。

```
1  @RestController
2  @AllArgsConstructor
3  public class ExceptionController {
4      private final ExceptionService exceptionService;
5
6      @GetMapping("/test/{id}")
7      public Result<String> getInfo(@PathVariable int id) {
8          if (id == 1) {
9              exceptionService.unAuthorizedError();
10         } else if (id == 2) {
11             exceptionService.systemError();
12         }
13         return Result.ok("请求成功");
14     }
15
16     @PostMapping("/user")
17     public Result<User> createUser(@Valid @RequestBody User user) {
18         return Result.ok(user);
19     }
20
21 }
```

4.1.4. 测试结果

可以对各个字段按照规则校验，并且做友好的返回。

POST ▼ http://127.0.0.1:8888/user 🕒 📤 发送

Query Body Path Headers Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ json ☐ xml ☐ raw ☐ binary

```
1 {
2   "username": "",
3   "password": "",
4   "age": 0,
5   "phone": ""
6 }
```

Body Cookies Headers

   JSON 200 54 ms 54 B

```
1 {
2   "code": 400,
3   "msg": "用户名不能为空",
4   "data": null
5 }
```

POST ▼ http://127.0.0.1:8888/user 🕒 📤 发送

Query Body Path Headers Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ json ☐ xml ☐ raw ☐ binary

```
1 {
2   "username": "aaa",
3   💡 "password": "",
4   "age": 0,
5   "phone": ""
6 }
```

Body Cookies Headers

   JSON 200 12 ms 51 B

```
1 {
2   "code": 400,
3   "msg": "密码不能为空",
4   "data": null
5 }
```

POST ▼ http://127.0.0.1:8888/user 🕒 📤 发送

Query Body Path Headers Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ json ☐ xml ☐ raw ☐ binary

```
1 {
2   "username": "aaa",
3   "password": "111",
4   "age": 0,
5   "phone": ""
6 }
```

Body Cookies Headers

   JSON 200 5 ms 55 B

```
1 {
2   "code": 400,
3   "msg": "年龄不能小于1岁",
4   "data": null
5 }
```

POST ▼ http://127.0.0.1:8888/user 🕒 📤 发送

Query Body Path Headers Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ json ☐ xml ☐ raw ☐ binary

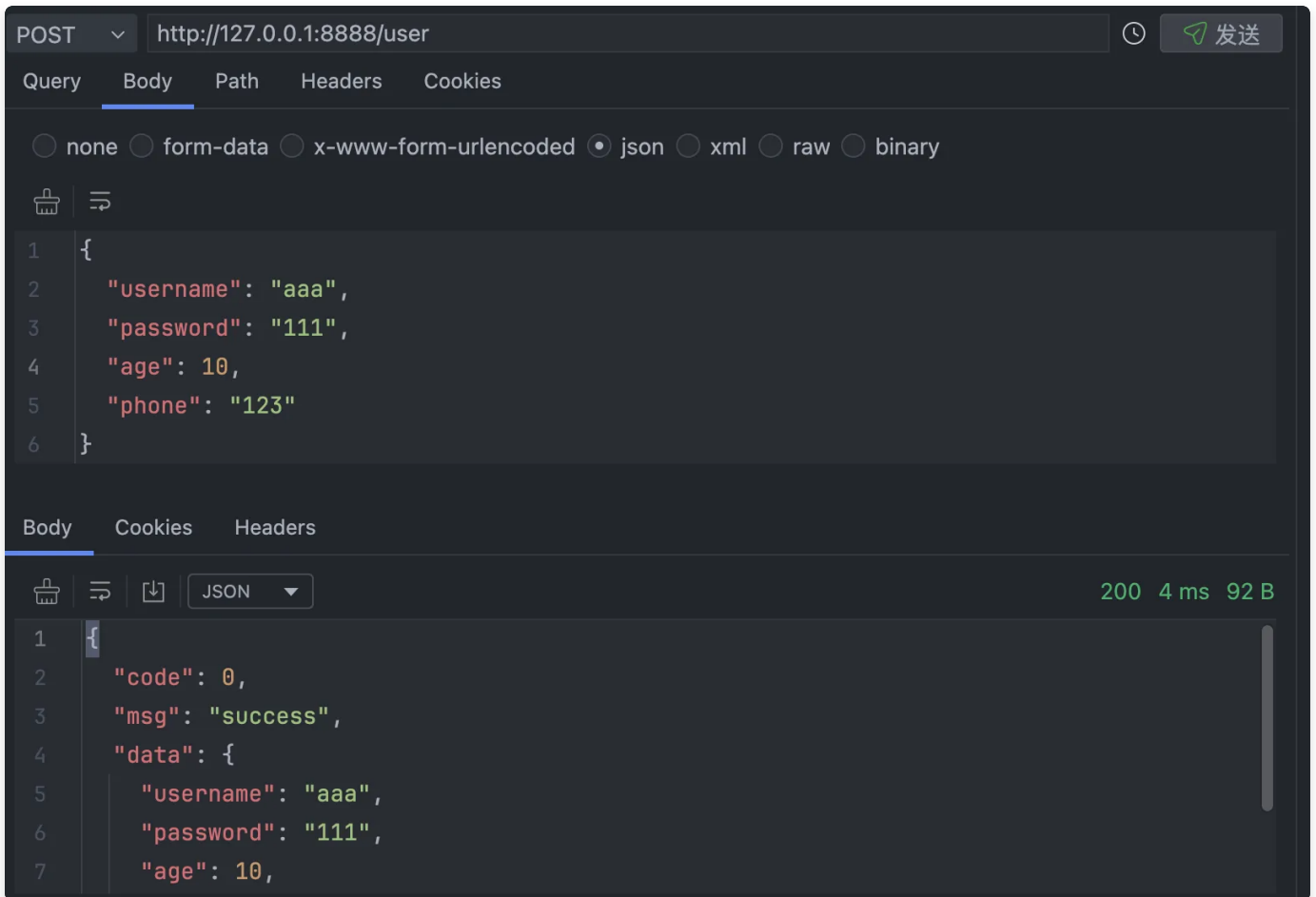
```
1 {
2   "username": "aaa",
3   "password": "111",
4   "age": 10,
5   "phone": ""
6 }
```

Body Cookies Headers

   JSON 200 4 ms 54 B

```
1 {
2   "code": 400,
3   "msg": "手机号不能为空",
4   "data": null
5 }
```

这里只对手机号非空做了校验。



4.2. 自定义注解

4.2.1. 编写自定义注解



这是一个基于 Bean Validation (JSR-303/JSR-380, 如 Hibernate Validator 实现) 的自定义约束注解, 用于校验“手机号”字段。逐项说明如下:

1. `@Target({ElementType.FIELD})`

仅能标注在字段上 (如 DTO/实体的属性)。若需要支持方法参数、记录组件等, 可扩展为 `ElementType.PARAMETER`, `ElementType.METHOD`, `ElementType.ANNOTATION_TYPE`, `ElementType.RECORD_COMPONENT` 等。

2. `@Retention(RetentionPolicy.RUNTIME)`

注解在运行时仍然可见, 校验框架在运行时读取它并执行校验逻辑 (若是 CLASS/SOURCE 级别将无法参与校验)。

3. `@Constraint(validatedBy = PhoneValidator.class)`

声明这是一个约束注解, 并指定其校验器类型。`PhoneValidator` 需要实现 `ConstraintValidator<Phone, String>` (或其他被校验类型), 在 `isValid` 方法中写入具体判定规则 (如正则匹配)。

4. `public @interface Phone { ... }`

定义注解本身, 并按规范提供以下三个成员 (均为 Bean Validation 约束的标准元素):

- `String message() default "手机号格式不正确";`

默认的**错误提示消息**。可用占位或国际化键 (如 `"${validatedValue}"` 或 `"{phone.invalid}"`), 也可在 `ValidationMessages.properties` 中覆盖。

- `Class<?>[] groups() default {};`

校验分组。用于在不同场景 (如创建/更新) 启用不同的约束集合。

- `Class<? extends Payload>[] payload() default {};`

承载元数据, 很少直接使用, 常用于把约束归类 (如严重级别) 供校验结果处理器识别。

4.2.2. 对应的校验器

```
1 public class PhoneValidator implements ConstraintValidator<Phone, String>
2 {
3     // 手机号正则表达式
4     private static final String REGEX_PHONE = "^1[3456789]\\d{9}$";
5
6     @Override
7     public void initialize(Phone phone) {
8     }
9
10    @Override
11    public boolean isValid(String value, ConstraintValidatorContext context) {
12        if (StringUtils.isBlank(value)) {
13            // 忽略空值, 由@NotBlank注解校验
14            return true;
15        }
16        return Pattern.matches(REGEX_PHONE, value);
17    }
18
19 }
```


这是基于 Bean Validation 的自定义约束校验器实现，用于校验中国大陆手机号码。逐行解析如下：

- `public class PhoneValidator implements ConstraintValidator<Phone, String>`

声明该类是 `@Phone` 注解的校验器，针对被校验类型 `String`。校验框架（如 Hibernate Validator）在运行时会调用它的 `isValid` 方法判断合法性。

- `private static final String REGEX_PHONE = "^1[3456789]\\d{9}$";`

定义手机号正则：

- `^` 与 `$`：整串匹配。
- `1`：中国大陆手机号以 1 开头。
- `[3456789]`：第二位号段限定为 3/4/5/6/7/8/9。
- `\\d{9}`：后续 9 位数字。

合起来即 11 位的常见大陆手机号格式。

- `initialize(Phone phone)`

初始化钩子。此处留空，表示注解上没有需要提前读取的动态参数（例如：可配置的正则、是否允许空值等）。留空是完全可以的。

- `isValid(String value, ConstraintValidatorContext context)`

核心校验逻辑：

- a. `StringUtils.isBlank(value)` 返回 `true` 时直接放行（返回 `true`）。
含义：空值交给 `@NotBlank` 等注解处理，本注解只负责“格式”而不负责“必填”。这是职责分离的良好实践。
- b. 非空时，使用 `Pattern.matches(REGEX_PHONE, value)` 执行正则匹配，匹配成功返回 `true`，否则返回 `false`。

4.2.3. 使用

在实体类的 `phone` 字段加上该注解即可，该注解内置了默认的提示信息。

```
1 @NotBlank(message = "手机号不能为空")
2 @Phone
3 private String phone;
```

测试结果，可以根据自定义的规则校验手机格式。

POST http://127.0.0.1:8888/user

Query Body Path Headers Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ json ☐ xml ☐ raw ☐ binary

```
1 {
2   "username": "aaa",
3   "password": "111",
4   "age": 10,
5   "phone": "123"
6 }
```

Body Cookies Headers

JSON

200 16 ms 57 B

```
1 {
2   "code": 400,
3   "msg": "手机号格式不正确",
4   "data": null
5 }
```